Question 2

a) In Spring Framework, Inversion of Control (IoC) is a principle in software engineering by which the control of objects or portions of a program is transferred to a container or framework as shown in Figure 4. It is most often used in the context of object-oriented programming such as Java.

[11 marks]

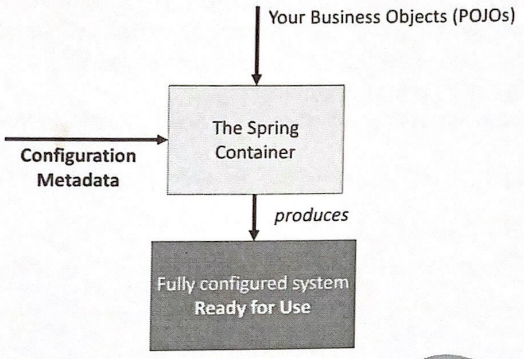


Figure 4

i) Based on the information in Figure 4, explain how the container in Spring allows the system to be fully configured and is ready for the application deployment and provide TWO examples of the configuration metadata

(4 marks)

ii) A bean XML definition "**module.xml**" and the Java class "**moduleBean.java**" using the Setter Dependency Injection are shown in Figure 5 with the bean ID = "**moduleBean**" and the property name = "**subject**". Now that the **moduleBean** is defined in both the XML definition and the Java class, write the **main()** method of class **MyModuleMain** as the main ***Application Context*** to load the configured bean.

(7 marks)

```
//module.xml
<bean id="StudentBean"  class="org.module.moduleBean">
      <property name="module" value="Middleware"/>
</bean>


//moduleBean.java
package org.module;
public class moduleBean {
      String module;
      public String getModule()
            { return module; }
      public void setModule(String module)
            { this.module = module; }
}
```

Figure 5

Q2a)

i) (Basic level)

The container in Spring gets its instructions on what objects to instantiate, configure, and assemble by reading configuration metadata provided[1 mark], and to take control of the flow of a program and make calls to customised code [1 mark].

The configuration metadata can be represented either by;

[NOTE: Any TWO of the following is correct - 1 mark each and 2 marks in total]

- XML, [1 mark].
- Java annotations, [1 mark].
- Java code. [1 mark].

ii) (Advanced level)

```
//myModule.java
package org.module;
public class MyModuleMain { [1 mark]
   public static void main(String[] args) { [1 mark]
      ApplicationContext appCon = new
      ClassPathXmlApplicationContext("module.xml"); [2 marks]
      moduleBean mod=(moduleBean)appCon. getBean("moduleBean"); [2 marks]
      mod.getName();[1 mark]
}
```

Solutions

b) Spring Java provides two types of containers: ***BeanFactory*** and ***ApplicationContext***. Generally, the Spring container is responsible for instantiating and managing the lifecycle of Spring beans.

[12 marks]

i) A Java class named **`StudentBean.java`** is shown in **Figure 4**. Configure a student bean XML definition with the property name = "**`name`**" and value = "**`Alex`**" as **`applicationContext.xml`** and add into the classpath accordingly [Note: include the configuration with **`<bean></bean>`** tag only].

(5 marks)

ii) From i), now that the Student bean is defined in the application context, write the **`main()`** method of class **`MyStudent`** to load the configured bean.

(7 marks)

```
//StudentBean.java

package org.example.student;

public class StudentBean {
      String name;
      public String getName()
            { return name; }
      public void setName(String name)
            { this.name = name; }
      public void displayInfo()
            { System.out.println("Hello: "+ name); }
}
```

Turn over until you reach the "END OF PAPER" line

Question 2

a) Describe the `Joinpoint` and `Pointcut` terminologies for Spring aspect-oriented programming (Spring AOP). **[4 marks]**